ParaDEVS Library

Fast branch copies: a proposal for speeding up the engine

ParaDEVS Library workspace, Binghamton

October 16, 2026 (proposal only: no engine code has been changed)

How to read this proposal. Page 3 is a one-page summary. Section 3 answers your question about the model being "half way through its transition". Every later section opens with a green "In short" box. Terms are defined in the glossary (Appendix B). Nothing in `src/main` has been changed; all measurements come from a prototype in `src/sandbox` (Appendix A).

ParaDEVS documents. The entry point is **Start Here**, a reading guide that links every document.

- **Build Report**: what has been built, how it was verified, what is next
- **How-To Guide**: setup, tutorials, API reference, troubleshooting
- **Architecture Guide**: how the library works and how to read results
- **RL Plan**: reinforcement-learning plan, analysis and results
- **Control Spec**: action selection: Para.decide, graph-mode planning, value and policy iteration
- **Hybrid Study**: which draws to enumerate or sample, and how cost grows with lookahead
- **Fast-Copy Proposal** (this document): why the engine replays steps, where time goes, how to speed it up

Contents

1 At a glance **3**

2 How a departure is run today **3**

3 Why the half-way model cannot be reused **4**

3.1 What "half way through its transition" means 4

3.2 Why not pause instead of throwing? 5

3.3 Consequence: replay is the right design, and the copy is what to speed up 5

4 Where the time goes **6**

4.1 Method 6

4.2 Profile under today's serialization 6

5 Option B: a generic fast copy, no model changes **7**

5.1 The idea . . . 7

5.2 A cheaper merge key . . . 8

5.3 A cheaper serialization (for the fallback) . . . 8

6 Option A: an opt-in hand-written copy **8**

7 Option C: stop doing work that is thrown away **9**

7.1 Continue with the first option . . . 9

7.2 The suspension exception . . . 9

7.3 Ideas measured or considered and not proposed . . . 9

8 Measured speed-ups **10**

9 Risks and safeguards **11**

9.1 What can go wrong . . . 11

9.2 The verification mode . . . 11

9.3 Thread safety, for a later parallel expansion . . . 12

10 Recommendation **12**

11 What you need to decide **13**

A Reproducing the measurements **14**

B Glossary **15**

1 At a glance

The problem	One departure (one run of one event) costs about 70–90 µs on the FrozenLake walker and about 250 µs on baseball. The model's own code is only 2–5% of that. **About 95% is copying**: turning the saved bytes back into objects (70–80%) and saving the result as bytes again (15–25%).
Your question	The model cannot be reused after a draw suspends it, because it is stopped in the middle of its transition and Java cannot resume it. Even if it could, following m outcomes needs m independent copies of the paused run, and a paused run cannot be copied. So *replaying from a clean copy is required for branching.* The speed-up has to come from making the copy cheap (Section 3).
What we measured	A prototype engine in `src/sandbox`, checked against the real engine (same values, departures and merges), with five ways of copying a branch.
Main result	Keep each finished branch as a live Java object and copy it directly in memory instead of through bytes: **20–25× faster on FrozenLake and 9–20× on baseball**, with no change to any model. A hand-written copy method (opt-in) reaches **28–35× and 29–90×**.
Also found	(1) The merge key costs 6% today and becomes the largest cost once copies are fast; a cheaper key builder fixes most of it. (2) 6–21% of runs are thrown away (the "discovery" run before each draw); they can be avoided by letting that run continue with the first outcome. (3) Live objects use *less* memory than bytes (MdpWalker: 280 instead of 1,550 bytes per branch).
The risk	A wrong copy gives wrong answers silently. The prototype includes a verification mode that compares every fast copy with a serialization copy and catches both kinds of mistake we tried, on the first copy or first run.
Recommendation	Phase 1 (generic fast copy, cheaper keys, verification mode in tests): about 3 days, about 20×. Phase 2 (opt-in `ParaCopyable`): about 1 day, a further 1.5–4× on large models. Phase 3 (no discovery run): about 2 days, a further 10–25%. The 10-minute comparison study (`CompareNStep`) should drop to well under a minute.

Decision needed
Section 11 lists four points for you to decide before any engine change: whether to adopt Phase 1 (and as the default or as an opt-in), whether to add the `ParaCopyable` interface to the public API, whether Monte Carlo runs may change their random streams (Phase 3), and whether the verification mode should run in every acceptance test.

2 How a departure is run today

In short
Each branch is kept as serialized bytes. Every run of an event first rebuilds the whole model from those bytes, runs the event, and, if it finished, saves the model to bytes again. A run that reaches a draw stops, is thrown away, and the event is re-run from the bytes once per outcome.

The engine (`ParaEngine.runLocal`) does these steps for every departure:

1. **Restore**: `Snapshots.restore` deserializes the parent branch's bytes into a fresh object graph: the model, its ports, its MS4 simulator, and for a coupled model all children and the routing tables.
2. **Wrap**: `ModelNode.of` builds the small driver objects that run the model in a fixed order.
3. **Run** the event with the values supplied so far. If the model calls `Para.choose` for a value that is not supplied yet, `Para` throws a `Suspension`; MS4 wraps it in its own exception; the engine catches it and records the pending law. The half-run model is discarded.
4. If the event finished: fold the outputs into the observation (**record**), build the merge key (**key**, only with safe merging), and **save** the model to new bytes.

So an event with one draw and b outcomes is run $1 + b$ times: one *discovery* run that finds the law and is thrown away, then one replay per outcome, each from a fresh restore (Figure 1).

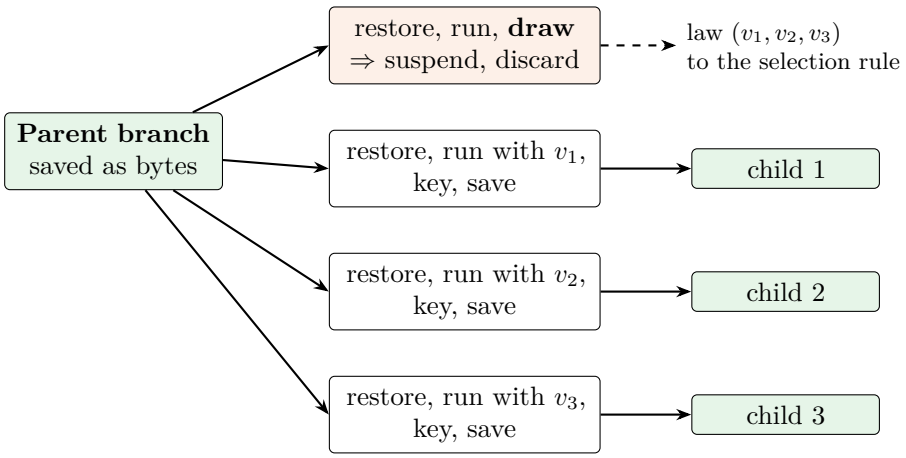


Figure 1: One event with one draw and three outcomes: four runs, four restores, three saves. The first run only discovers the law.

3 Why the half-way model cannot be reused

In short
When a draw suspends a run, the model object is in the middle of its transition: some fields are already updated, others are not, and part of the state lives only in Java's call stack, which the suspension destroys. Java cannot pause a method and resume it later. Even with real pausing, branching into m outcomes needs m independent copies of the paused run, and a paused run (its stack) cannot be copied. So branching must restart from a clean copy of the state before the event. What *can* be saved is the cost of that copy, and the one wasted run.

3.1 What "half way through its transition" means

Take the moment `Para.choose` throws. At that moment:

- **The model's fields are partly updated.** A transition is ordinary Java code that assigns fields one after another. Everything assigned before the draw is already changed; everything after it is not. In `MdpWalker` in `SEPARATE` mode, the action has been drawn and the state not yet; in `Batter` the input has been read and the hit not yet chosen.
- **MS4's bookkeeping has moved on.** Before calling the model, MS4's simulator has already set its current time and fired its transition event; after the call it would set the last and next event times, but that part never runs. In a coupled model, children earlier in the order may have finished their transitions while later ones have not started.
- **Some of the state exists only on the call stack.** Local variables (the drawn action `a`, the row of probabilities, the coordinator's in-box of messages being routed) live in the stack frames of the methods that are running. The suspension is an exception: it unwinds the stack, and those frames are gone.

The object that is left is neither the state before the event nor the state after it. It cannot be saved as a branch and it cannot be continued, so the engine throws it away.

3.2 Why not pause instead of throwing?

Java has no operation that stops a method in the middle and continues it later from the same point. There are two ways to get something like it, and both fail for the same reason:

- **A thread per run**, blocked inside `Para.choose` until the engine supplies the value. This does pause the run with its stack intact.
- **Continuations** (Java's virtual threads, "Project Loom"): the same idea, cheaper.

Both give *one* paused run. A draw with three outcomes needs *three* runs that continue from that point, each with a different value, and each changing its own copy of the model. That needs three copies of the paused run: its stack frames *and* every object they point to, including the model. Java cannot copy a thread or a continuation (Loom continuations are one-shot by design), and copying the model objects is exactly the cost we are trying to avoid. Copying a whole operating-system process (Unix `fork`) is not possible inside a JVM either.

3.3 Consequence: replay is the right design, and the copy is what to speed up

Replay from a clean copy is the standard way to branch a program that cannot be cloned (the paper calls it the derandomized departure). Its extra cost is re-running the event code up to the draw, and that code is cheap: the model's own code is 2–5% of the time today (Table 1). The expensive part is the copy. Hence the proposal:

1. make the copy cheap (Sections 5 and 6);
2. stop paying for work that is thrown away (Section 7). The half-way run cannot serve m outcomes, but it can serve *one*: if it simply continues with the first outcome instead of stopping, it becomes the first child and nothing is wasted.

4 Where the time goes

In short
Restoring from bytes takes 70–80% of a departure and saving takes 15–25%. The model's own code takes 2–5%. The merge key takes 6% on baseball with merging. Runs that end at a draw (and are discarded) take 5–19% of the time.

4.1 Method

The prototype `sandbox.fastcopy.ProtoEngine` is a copy of `ParaEngine` (without input traces and acceptors, which these workloads do not use) with `System.nanoTime` timers around each step. Before any timing, the harness checks that it gives the real engine's value, number of departures and number of merges for every strategy (FrozenLake from six start states with and without merging; baseball with 4 and 6 plate appearances). The workloads:

Workload	What it runs
FL exhaustive	`MdpWalker` on FrozenLake 8×8 (the ε-soft policy of `CompareNStep`), 6-step targets from start states 0, 10, 18, 33, 44, 57, every outcome, no merging.
FL exhaustive + merge	The same with safe merging (repeated 8 times for stable timing).
FL k=3 tree + merge	Monte Carlo with 3 samples per draw and safe merging, 20 seeds per start state: the kind of run that dominates `CompareNStep`.
BB 5 PA	Baseball (coupled model: `Batter` + `BallGame`), 5 plate appearances, no merging.
BB 6 PA + merge	Baseball, 6 plate appearances, safe merging (repeated 6 times).

All runs: JDK 17.0.15, `-Xmx512m`, two processors, Windows 11, on a machine shared with other jobs. Single numbers vary by about ±30% between runs; the ratios in Section 8 are stable.

4.2 Profile under today's serialization

Workload	Total	Restore	Save	Model	Key	Other	Bytes	Susp.
FL exhaustive	71.9	57.4 (80%)	10.6 (15%)	2.4 (3%)	0.0	1.6 (2%)	1,531	12.7%
FL exh. + merge	82.5	65.7 (80%)	11.6 (14%)	3.2 (4%)	0.3	1.8 (2%)	1,531	17.3%
FL k=3 + merge	72.8	58.2 (80%)	9.7 (13%)	3.0 (4%)	0.2	1.7 (2%)	1,531	21.3%
BB 5 PA	249	184 (74%)	58.8 (24%)	4.1 (2%)	0.0	2.4 (1%)	4,803	6.5%
BB 6 PA + merge	271	188 (69%)	56.4 (21%)	4.8 (2%)	16.3 (6%)	5.6 (2%)	4,806	12.0%

Table 1: Time per departure in microseconds (share of the total in brackets). "Model": the event's own code, including the throw when it suspends. "Other": wrapping, recording the outputs and the engine's bookkeeping. "Bytes": size of one saved state. "Susp.": share of runs that end at a draw (the discarded discovery runs).

What this shows:

- **Copying is the whole cost.** The state of `MdpWalker` is four numbers, but its serialized form is 1,531 bytes, because it includes the class descriptions of the model, its ports, its

MS4 simulator and their lists, written out afresh in every snapshot. Reading it back means looking up each class by name and building every object through the serialization machinery. Baseball's snapshot (4,800 bytes) also carries both batting tables and MS4's routing tables.

- **Restoring costs 5 times more than saving**, mostly class lookups and object construction.
- **The discovery runs** (one per departure that draws) cost 5–19% of the time today: 12–21% of runs on FrozenLake, 6–12% on baseball.
- **The merge key** on baseball costs 16 µs: `StateKeys` converts both batting tables into lists of boxed numbers on every completed departure, and recomputes field lists by reflection each time. For `MdpWalker` it is negligible because the model supplies its own key (`ParaStateKey`).
- **Wrapping, recording and the engine's bookkeeping** are 1–2% (`ModelNode.of` alone is under 1 µs). Re-creating the simulator wrappers is not worth optimizing.

5 Option B: a generic fast copy, no model changes

In short
Keep each finished branch as the live Java object that produced it (it is never touched again), and start each run from an in-memory deep copy made by reflection instead of from bytes. This gives 20–25× on FrozenLake and 9–20× on baseball, works for any model the current code supports, and falls back to serialization for anything it does not understand.

5.1 The idea

Today a finished run is saved to bytes and the object is dropped. Instead, the finished object itself can be stored as the branch state, as long as nothing ever changes it again. Every later run of that branch starts from a copy. This removes the save step completely and replaces "bytes to objects" by "objects to objects".

The copy (`sandbox.fastcopy.ReflectCopy`) visits the object graph once and produces the same graph that a serialize-deserialize round trip produces:

- strings, boxed numbers, enums and classes are shared (they cannot change);
- arrays are copied element by element;
- JDK lists, maps, sets and priority queues are rebuilt through their normal methods, so hash tables are rebuilt for the new keys (MS4's routing table is a hash map keyed by port objects, so a field-by-field copy would break it);
- every other object must be serializable without custom serialization code; it is created the way serialization creates it (no constructor run) and its fields are copied; fields marked `transient` are left empty, exactly as serialization leaves them;
- what is learnt about a class (its fields, how to create it) is computed once and cached.

Anything else (a class with its own `readObject`, an unusual JDK collection) makes that model class fall back to today's serialization, so no model stops working.

5.2 A cheaper merge key

Once copies are cheap, the merge key becomes the largest cost on baseball. The prototype `FastKeys` builds *exactly the same equalities* as `StateKeys`, but caches each class's field list, keeps a primitive array as one wrapped copy instead of a list of boxed numbers, and caches the key's hash code so the merge table does not walk it again. On baseball with merging this cuts the key from 16 µs to about 4–5 µs per departure.

5.3 A cheaper serialization (for the fallback)

For models that must stay on serialization, two small changes make it 1.3–1.7× faster: caching the name-to-class lookup that deserialization does for every class description, and reusing the output buffer (`SERIAL_TUNED` in the tables). A faster third-party serializer (Kryo, FST) would add a dependency and has the same correctness questions as the reflective copy, for less gain, so it is not proposed.

6 Option A: an opt-in hand-written copy

In short
A model can implement `ParaCopyable` and copy its own fields in a few lines; the engine copies MS4's bookkeeping itself. Read-only data (tables) can then be shared instead of copied. This is the fastest option, especially for coupled and larger models (baseball up to 90×), but a mistake in the copy gives wrong answers, so it needs the verification mode of Section 9.

```
public interface ParaCopyable {
    /** A new, independent model with the same state. Copy only this model's own fields;

        the engine copies MS4's event times. Read-only data may be shared. */
    AtomicModel paraCopy();
}

public class FastMdpWalker extends MdpWalker implements ParaCopyable {
    @Override public AtomicModel paraCopy() {
        FastMdpWalker c = new FastMdpWalker(mdpName, startState, steps, mode);
        c.s = s; c.left = left; c.done = done; c.sigma = sigma;
        return c;
    }
}
```

The model builds a fresh instance through its normal constructor (which creates ports and the MS4 simulator) and sets its state fields. The engine then copies the MS4 bookkeeping (the simulator's last, next and current times and its flags) for the model and, for a coupled model, for every child in order (`sandbox.fastcopy.Bookkeeping`). For baseball the copy shares the two batting tables, which never change; the generic copy has to copy them because it cannot know that. Models that do not implement the interface use the generic copy, or serialization.

The prototype adds the hook to the two example models (`sandbox.fastcopy.FastModels`): 4 lines for `MdpWalker`, about 15 for baseball.

7 Option C: stop doing work that is thrown away

In short
The discovery run before each draw is pure waste: 6–21% of all runs. If, at a draw with no supplied value, `Para` asks the engine for the options right there and lets the run continue with the first one, that run becomes the first child and only the other outcomes are replayed. This removes every discarded run (and the exception that MS4 builds for it) and gives a further 10–25%, with identical results for exhaustive runs.

7.1 Continue with the first option

This is how the half-way run *can* be reused: not by pausing it, but by never stopping it. When `Para.choose` reaches a value that has not been supplied, it calls back into the engine, which runs the selection rule, queues the other selected values for replay (each with the values used so far), and returns the first value. The run goes on and finishes as the first child with weight $w\alpha_1$. A second draw in the same event works the same way. An event with one draw and b outcomes is then run b times instead of $b + 1$.

The prototype implements it with a small addition to a *copy* of `Para` in `src/sandbox/patches` (not compiled into the library) and an `inline` switch in `ProtoEngine`. Measured:

Workload	Departures today	With inline	Result
FL exhaustive	30,358	26,500 (−13%)	same value
FL exhaustive + merge (8×)	35,016	28,944 (−17%)	same value
FL k=3 + merge	24,927	19,629 (−21%)	statistically equivalent (different random stream)
BB 5 PA	18,200	17,023 (−6%)	same value
BB 6 PA + merge (6×)	17,946	15,786 (−12%)	same value

The saving equals the share of runs that used to suspend. Because the selection rule is now called while the branch is running instead of afterwards, Monte Carlo runs draw their random numbers in a different order: their results are equally valid but not identical to today's for the same seed. Exhaustive runs give the same answers (to the last few bits of floating-point summation order).

7.2 The suspension exception

MS4's simulator catches every exception from model code and wraps it in a new MS4 exception (`DEVSRuntimeException`), which records a full stack trace. That costs about 1.4–1.8 µs per suspension (a stack-less exception costs 0.03 µs). It is invisible today but becomes about 5–10% of a departure once copies are fast. Continuing with the first option removes suspensions altogether, so this cost disappears with it.

7.3 Ideas measured or considered and not proposed

- **Re-using the wrappers** (`ModelNode`): under 1 µs per departure; not worth the complexity.

- **Letting the last outcome run on the parent itself** instead of a copy: it saves one copy per draw, but if that run reaches a second draw the parent is destroyed while still needed. Unsafe in general for a small gain.
- **Threads or continuations to pause runs**: cannot branch (Section 3).

8 Measured speed-ups

In short
Wall-clock speed-ups against today's serialization, all with identical results: the generic copy with the cheaper key gives 15–26×; the hand-written copy gives 28–89×; continuing with the first option adds 7–40% on top.

Workload	Today (ms)	Tuned serial.	Generic copy	Generic + key	Hook + key	Hook + key + inline	Verify mode
FL exhaustive	2,105	1.7×	25×	24×	35×	41×	0.7×
FL exh. + merge	3,060	1.5×	20×	26×	28×	35×	1.1×
FL k=3 + merge	1,922	1.5×	20×	18×	28×	39×	0.4×
BB 5 PA	6,460	1.6×	16×	20×	89×	95×	1.2×
BB 6 PA + merge	4,982	1.3×	9×	15×	29×	33×	1.0×

Table 2: Speed-up of the total run time over today (best of 2 runs for the serialization strategies, best of 7 for the fast ones; timers off). "Hook" uses the hand-written `paraCopy()`; "key" is the cheaper key builder; "inline" continues with the first option. "Verify mode" runs the hook and checks it against serialization on every copy (Section 9).

Time per departure after the change. With the hand-written copy, a FrozenLake departure takes about 2–3 µs instead of 70–90 µs, and a baseball departure about 4–10 µs instead of 250–280 µs. What remains is roughly a third each for the copy, the model's own code, and the key plus the engine's bookkeeping. The generic copy costs about 1.5 µs for `MdpWalker` and 13–18 µs for baseball, because it also copies the batting tables and rebuilds MS4's routing maps.

Memory. Live objects are smaller than their bytes, because the bytes repeat every class description:

Stored branch state	Bytes (today)	Generic copy	Hand-written copy
`MdpWalker`	1,553 B	281 B	313 B
Baseball game	4,812 B	3,682 B	2,489 B

The 10-minute comparison study. `CompareNStep` runs the same walker as the FrozenLake workloads, and nearly all its time is departures. With a 20–30× faster departure it should take roughly 20–40 seconds instead of about 10 minutes. This is an estimate from the profile; it will be measured once Phase 1 exists.

9 Risks and safeguards

In short
A fast copy that misses a field, or shares something that changes, gives wrong answers without any error. A verification mode that makes every copy twice (fast and by serialization) and compares them catches both mistakes on the first event. It should run in every acceptance test. Live branch states must be treated as frozen, which also makes later parallel expansion safe.

9.1 What can go wrong

1. **An incomplete copy**: `paraCopy()` forgets a field. The copy starts in the wrong state. In the prototype, forgetting `left` in `MdpWalker` makes the walk stop at once: the 6-step return from state 55 becomes 0.000 instead of 0.565, with 2 departures instead of 1,570, and *no error*.
2. **A shared mutable object**: `paraCopy()` shares an array or list that the model later changes. Each copy starts correctly, but running it changes the frozen parent and every later replay of it.
3. **The generic copy and unusual classes**: a class whose serialization does something special (`readObject`, `readResolve`) or a JDK collection the copier does not know. The copier detects these and falls back to serialization; this is safe but slow.
4. **Changing a frozen state**: anything that writes to a stored branch object (a model that caches something in a getter, code that holds on to a model after its run) breaks every branch that shares it. Today the bytes make this impossible.
5. **JDK internals**: the generic copy creates objects the way serialization does, through `sun.reflect.ReflectionFactory` (module `jdk.unsupported`). It is available in current JDKs and used by many libraries, but it is not a standard API.

9.2 The verification mode

Proposed as `ParaSpec.verifyCopies(true)`, and switchable for a whole test run with a system property, so that `scripts/build_library.sh` can run every acceptance test once with it. For each departure it:

1. makes the fast copy and a serialization copy of the parent, and compares them field by field over the whole model tree, including MS4's event times. It ignores `ParaStateKey`, since a hand-written key can be as incomplete as a hand-written copy. A difference stops the run with the model, field and both values;
2. serializes the parent before the run and again after it; if the bytes differ, the copy shared something the run changed, and the run stops with that message.

In the prototype (`HOOK_VERIFY`) both deliberate bugs are caught immediately:

```
paraCopy() is incomplete at MdpWalker: MdpWalker.left is 0 in the fast copy but 6 in
    the
  serialization copy
running a copy changed its parent MdpWalker: paraCopy() shares mutable state between
    the
```

```
  copy and the original
```

Verification costs about what today's engine costs (0.4–1.2× today's speed in Table 2), so it is fine in tests but not meant for production runs. In addition, each acceptance test should compare a run with fast copies against a run with serialization: same distribution, same departures, same merges. The prototype harness already does this for every strategy.

9.3 Thread safety, for a later parallel expansion

Fast copies do not stop the engine from expanding branches in parallel later, if these rules hold:

- **Frozen means read-only.** A stored branch is only ever read (copied, keyed). Several threads may copy the same parent at once. `paraCopy()` must not change the object it copies.
- **Hand-over between threads** goes through the engine's own concurrent queues, which makes the frozen object safely visible to the other thread.
- **Caches are concurrent.** The copier's class plans, the key builder's field lists and the class-lookup cache are concurrent maps holding data that never changes once computed.
- **Draw context**: `Para`'s context is already per thread. The selection rule's random generator is shared today; a parallel engine needs one generator per branch (split from the seed) to stay reproducible.
- **Constructors in `paraCopy()`** run model and MS4 constructor code, which might touch static state (MS4's coupling class has a static field). This needs checking before parallel use. The generic copy runs no constructors.

10 Recommendation

In short
Do Phase 1 first: it needs no model changes and gives about 20×. Add the opt-in hook and the first-option change afterwards, each on its own, with the verification mode in the tests from the start.

Phase	What changes	Effort	Expected gain
1. Generic fast copy	Branches hold frozen live objects; runs start from the reflective copy, with serialization as fallback (and the class-lookup cache for it); cheaper merge key in `StateKeys`; verification mode and fast-versus-serialization comparison in every acceptance test; architecture guide updated.	about 3 days	15–25× on these models; `CompareNStep` from about 10 min to under a minute
2. `ParaCopyable` (opt-in)	Public interface plus engine support and MS4 bookkeeping copy; hooks on `MdpWalker` and baseball; a how-to section on writing a correct copy.	about 1 day	a further 1.5× (FrozenLake) to 4× (baseball)
3. Continue with the first option	`Para` asks the engine for options at an unsupplied draw; the expansion loop replays only the other options.	about 2 days	a further 10–25%; removes all suspensions
Quick win (any time)	Only the class-lookup cache and buffer reuse in `Snapshots`.	1–2 hours	1.3–1.7×

Every phase keeps today's results: exhaustive answers, departures and merges stay the same (Phase 3 changes the departure count, by design, and the Monte Carlo random stream). All 99 checks would be re-run after each phase, plus the same checks with verification switched on.

11 What you need to decide

In short
Four choices, each with a recommended answer.

1. **Adopt Phase 1, and make it the default?** Recommended: yes, as the default, with serialization still used automatically for models the copier cannot handle and a switch to force serialization. It relies on `sun.reflect.ReflectionFactory` (Section 9, item 5). The alternative is to require a no-argument constructor in every class of a model, which MS4-generated code does not always have.
2. **Add `ParaCopyable` to the public API?** Recommended: yes, as an opt-in for models where speed matters, documented together with the verification mode. It is the only option that puts correctness in the model author's hands.
3. **Accept a new Monte Carlo random stream (Phase 3)?** After Phase 3, a Monte Carlo run with a given seed gives a different, equally valid sample than today. Re-running the RL studies would then change their numbers within their error bars. Recommended: yes, and record it in the decision log.
4. **Run the verification mode in every acceptance test?** Recommended: yes. It roughly doubles the test time (a few minutes).

A Reproducing the measurements

Everything is in `src/sandbox/java/sandbox/` and marked as a measurement prototype; nothing in `src/main` uses it. From the repository root in Git Bash, after `bash scripts/build_library.sh`:

```
javac -cp "build/library/classes;lib/ms4-core.jar" -d build/proto-patch \
      src/sandbox/patches/paradevs/Para.java       # only for the "inline" rows
java -Xmx512m -XX:ActiveProcessorCount=2 \
     -cp "build/proto-patch;build/library/classes;lib/ms4-core.jar" sandbox.
    ProfileEngine
```

Options: `-Dparts=123456` selects the parts (1 cross-check, 2 profiles, 3 speed-ups, 4 verification demo, 5 exception cost, 6 memory); `-Dwork=FL` or `-Dwork=BB` restricts the workloads; `-Dreps=2` sets the repetitions. Each part runs in under 3 minutes. Without `build/proto-patch` on the class path the inline rows are skipped.

File (in `src/sandbox/`)	Contents
`java/sandbox/ProfileEngine.java`	The harness: workloads, cross-check with the real engine, tables, demos
`java/sandbox/fastcopy/ProtoEngine.java`	Instrumented copy of the engine with the copy strategies
`java/sandbox/fastcopy/ReflectCopy.java`	Generic reflective deep copy (Option B)
`java/sandbox/fastcopy/FastKeys.java`	Cheaper merge key with the same equalities
`java/sandbox/fastcopy/ParaCopyable.java`, `Bookkeeping.java`	The opt-in hook and the MS4 bookkeeping copy (Option A)
`java/sandbox/fastcopy/FastModels.java`	Example models with `paraCopy()`, plus two deliberately wrong copies
`java/sandbox/fastcopy/Verify.java`	The verification check
`patches/paradevs/Para.java`	Copy of `Para` with the "continue with the first option" call-back (not compiled by the build)

B Glossary

Departure	One run of one event from a saved branch state; the unit counted in the run statistics.
Draw	One call to `Para.choose`: a random value with a known finite law.
Suspension	What happens when a draw's value is not yet supplied: `Para` throws, the run stops, and the engine learns the law.
Discovery run	The first run of an event, which only finds out the law of its first draw and is then discarded.
Replay	Running the event again from the parent's saved state, with the drawn values supplied.
Branch state	The saved model state of a branch. Today: serialized bytes. Proposed: a frozen live object.
Serialization	Java's built-in way of turning an object graph into bytes (save) and back (restore).
Deep copy	A copy of an object together with everything it refers to, so that changing one never affects the other.
Aliased object	An object referred to by both the original and the copy; harmless if it never changes, a silent bug if it does.
Reflection	Java's ability to read and write an object's fields by name at run time.
Frozen	A stored branch object that nobody changes any more; it is only copied and read.
`ParaCopyable`	Proposed opt-in interface: the model copies its own fields in `paraCopy()`.
MS4 bookkeeping	The event times and flags kept by MS4's simulator for each atomic model.
Merge key	The value `StateKeys` builds from a model's complete state; equal keys may be merged safely.
Continue with the first option	Proposed change: at an unsupplied draw, the run goes on with the first selected value instead of stopping, and only the other values are replayed.
Continuation / virtual thread	Java mechanisms that pause a running method; they cannot be copied, so they cannot branch.
Verification mode	Test setting in which every fast copy is checked against a serialization copy.
µs	Microsecond, a millionth of a second.